

Subject : Java 8

① Convert List<Employee> to map <Id, name>

Coding:

```
import java.util. Arrays;  
import java.util. List;  
import java.util. Map;  
import java.util. Stream. Collectors;
```

```
public class sample {  
    public static void main (String [] args)
```

```
{
```

```
    class Emp {
```

```
        int id;
```

```
        String name;
```

```
        Emp (int id, String name) {
```

```
            this. id = id;
```

```
            this. name = name;    } }
```

```
        List<Emp> l = Arrays.asList(  
            new Emp(1, "Priya"),  
            new Emp(2, "Divya"),  
            new Emp(1, "Vino") );
```

```
        Map<Integer, String> map = l. Stream(). collect  
            (Collectors. toMap(  
                e -> e. id,  
                e -> e. name, ));
```

```
        System. out. println (map);
```

```
    }
```

```
}
```

② Java 8

To find a 2nd largest number in a list

Coding:

```
import java.util.*;  
import java.util.stream.*;  
  
public class Large {  
    public static void main (String [] args)  
    {  
        List <Integer> list = Arrays.asList(10, 20, 50,  
                                                40, 30);  
  
        int HighSecond = list.stream()  
            .distinct()  
            .sorted (Collection.reverseOrder());  
            .skip(1)  
            .findFirst()  
            .get();  
  
        System.out.println ("Second Highest No: " +  
                                Second);  
    }  
}
```

Why:

- list.stream → Converts list to Stream
- distinct → removes duplicates
- sorted (Collections.reverseOrder()) → sorts in descending.
- skip(1) → skips the highest element
- find first → Get's the next element (2nd highest).

Subject: Collection

④ Program to merge two list of integers and remove any duplicate elements.

Answer:

```
import java.util. Arrays;  
import java.util. HashSet;  
import java.util. List;  
  
Public class collection {  
    public static void main (String[] args)  
    {  
        List<Integer> list1 = Arrays.asList (1, 2, 3);  
        List<Integer> list2 = Arrays.asList (2, 3, 4, 5);  
        HashSet<Integer> list = new HashSet<> ();  
        list.addAll (list1);  
        list.addAll (list2);  
        System.out.println (list);  
    }  
}
```

output:

[1, 2, 3, 4, 5]

⑤ Flight Reservation System:

Questions:-

i) I prefer HashMap collection framework for this.

ii) Coding:

```
import java.util.HashMap;

public class collection {

    public static void main (String[] args) {

        HashMap<Integer, String> pass = new HashMap<>();

        pass.put (105, "Divya, 22, Norway");
        pass.put (101, "Shobana, 20, Japan");
        pass.put (102, "Vino, 33, London");
        pass.put (103, "bala, 27, SouthKorea");
        pass.put (104, "Priya, 40, Australia");

        System.out.println (pass);
    }
}
```

iii) To get passenger details for a given ticket no:

Answer:

```
System.out.println (pass.get (102));
```

Output:

Vino, 33, London

iv) HashMap does not maintain the insertion order. So I will choose LinkedHashMap collection which maintains the order of Insertion.

④ Subject: Exception Handling

⑥ File Upload System:

Coding:

```
import java.io.File;  
public class FileUpload {  
    public static void main (String [] args)  
    {  
        String path = "document.pdf";  
        try {  
            File fi = new File(path);  
            if (!fi.exists()) {  
                throw new Exception("File doesn't exist");  
            }  
            if (!fi.getName().toLowerCase().endsWith(".pdf")) {  
                throw new Exception("Only pdf's are allowed");  
            }  
            int maxSize = 10 * 1024 * 1024;  
            if (fi.length() > maxSize) {  
                throw new Exception("file size is > 10 MB");  
            }  
            System.out.println("File uploaded successfully");  
        }  
        catch (Exception e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```

OOPS

⊕ E-Commerce Application

Coding:

```
abstract class {
```

```
    int p-id;
```

```
    String p-name;
```

```
    double price;
```

```
    public Product(int p-id, String p-name, double price)
```

```
{
```

```
    this.p-id =
```

```
    this.p-id = p-id;
```

```
    this.p-name = p-name;
```

```
    this.price = price;
```

```
}
```

```
    abstract double calculateDiscount();
```

```
} // public void display() {
```

```
    System.out.println("Product id: " + p-id);
```

```
    System.out.println("Product Name: " + p-name);
```

```
    System.out.println("Price: " + price);
```

```
    System.out.println("Discount: " + calculateDiscount());
```

```
    System.out.println("Final price: " + (price -  
        calculateDiscount()));
```

```
}
```

```
}
```

```
class Electronic extends Product {
```

```
    public Electronic(int id, String name, double price)
```

```
{
```

```
        super(id, name, price);
```

```
}
```

```
    double calculateDiscount() {
```

```
        return price * 0.10;
```

```
}
```

```
}
```

```

class cloth extends Product {
    public Cloth (int id, String name, double price) {
        Super (id, name, price);
    }
    double CalculateDiscount() {
        return price * 0.20;
    }
}

```

```

class Books extends Product {
    public Books (int id, String name, double price) {
        Super (id, name, price);
    }
    double CalculateDiscount() {
        return price * 0.30;
    }
}

```

```

public class EcommerceApp {
    public static void main (String[] args) {
        Product p1 = new Electronic (101, "Laptop", 50000);
        Product p2 = new Cloth (102, "Shirt", 1500);
        Product p3 = new Book (103, "Java", 850);
        p1.display();
        p2.display();
        p3.display();
    }
}

```

why I used this?

- 1) Encapsulation - Common product details
- 2) Inheritance - Electronic, Books, cloth inherit product class
- 3) Abstraction - Product (abstract class), CalculateDiscount()
- 4) Polymorphism - diff product type implements CalculateDiscount() differently,

8) Online Payment System:

Coding:

```
abstract class Payment {  
    String paymentDetail;  
    Payment(String paymentDetail) {  
        this.paymentDetail = paymentDetail;  
    }
```

```
    public String getPaymentDetails() {  
        return paymentDetail;  
    }
```

```
    abstract void processPayment();  
}
```

```
class CreditCard extends Payment {
```

```
    CreditCard(String details) {
```

```
        super(detail);
```

```
    void processPayment() {
```

```
        System.out.println("Payment made using credit  
Card");
```

```
    }
```

```
class UPI extends Payment {
```

```
    UPI(String detail) {
```

```
        super(detail);
```

```
    }
```

```
    void processPayment() {
```

```
        System.out.println("Payment made using  
UPI");
```

```
    }
```

```
}
```

```

class NetBanking extends Payment {
    NetBanking (String detail) {
        super (detail);
    }
    void processPayment() {
        System.out.println ("payment made using
                               Net Banking");
    }
}

```

```

public class DemoPayment {
    public static void main (String[] args)
    {
        Payment P1 = new CreditCard ("1234-xxxx");
        Payment P2 = new UPI ("user@upi");
        Payment P3 = new NetBanking ("SBI Account");
        P1.processPayment();
        P2.processPayment();
        P3.processPayment();
    }
}

```

why I use this Design?

- 1) Abstraction - hides payment implementation details
- 2) Encapsulation - protects sensitive payment info.
- 3) Inheritance - CreditCard, UPI, NetBanking inherit from Payment.
- 4) Polymorphism - Each payment method has it's own processPayment() implementation.